Media Transcription API

A Beginner's Toolkit for AI-Powered Speech-to-Text

Built with FastAPI · OpenAI Whisper · MySQL · Streamlit

Author	Peter Shako
Course	Foundations: Generative AI for Developers
School	Moringa School
Year	2026
GitHub	github.com/PeterShako/media-automation-api

1. Title & Objective

Getting Started with AI-Powered Transcription using FastAPI & Whisper

Technology Chosen

Building an AI-powered transcription API using **FastAPI** (a modern Python web framework) combined with **OpenAI Whisper** (a multilingual speech-to-text model), **MySQL** for persistent storage, and **Streamlit** for the user interface.

Why This Technology?

- FastAPI is one of the fastest-growing Python frameworks with automatic API documentation, type safety, and async support.
- OpenAI Whisper is freely available, runs locally (no API costs), and supports 90+ languages including Swahili.
- The combination of these tools reflects a real-world AI microservice architecture used in production systems.
- Streamlit allows building a functional UI in pure Python without any frontend knowledge.

End Goal

Build a working web application where a user can upload an audio or video file, have it transcribed by Whisper AI, view the result in a browser, and download it as a text file, all stored in a MySQL database.

2. Quick Summary of The Technology

FastAPI: A modern, high-performance Python web framework for building APIs. It uses Python type hints to auto-generate documentation and confirm data.

Real-world example: Netflix uses FastAPI for internal ML model serving. Uber uses it for microservices.

OpenAI Whisper: An open-source automatic speech recognition (ASR) model trained on 680,000 hours of multilingual audio. It runs locally on your machine, no API key or internet required.

Real-world example: Journalists use Whisper to transcribe interviews. Developers use it to add captions to videos automatically.

MySQL: The world's most popular open-source relational database. Stores structured data in tables with rows and columns.

Real-world example: Used by Facebook, Twitter, YouTube, and WordPress to store billions of records.

Streamlit: A Python library that turns Python scripts into shareable web apps in minutes. No HTML, CSS, or JavaScript required.

Real-world example: Data scientists use Streamlit to share ML model demos and dashboards with non-technical stakeholders.

SQLModel: **A** Python ORM (Object-Relational Mapper) that bridges FastAPI and SQLAlchemy. Define your database schema as Python classes.

Real-world example: Used alongside FastAPI in production APIs to manage database models with type safety.

3. System Requirements

Requirement	Minimum Version	Notes
Operating System	Windows 10/11, macOS 12+, Ubuntu	20.04+All platforms supported
Python	3.10+	3.12 recommended
MySQL Server	8.0+	Requires cryptography package for auth
MySQL Workbench	8.0+	For database management
FFmpeg	Any recent build	Must be added to system PATH
RAM	8GB minimum	16 GB
Storage	5GB free	Whisper models range from 140MB to 3GB
Code Editor	VS Code recommended	Any editor works

Python Packages Required

FASTAPI

Web framework uvicorn

ASGI server to run FastAPI sqlmodel

ORM for database models pymysql

MySQL driver cryptography

Required for MySQL 8 SHA2 authentication OpenAI-whisper

AI speech-to-text model Streamlit

Frontend UI framework requests

HTTP client for Streamlit to call the API.

4. Installation & Setup Instructions

Step 1: Clone the Repository

git clone https://github.com/PeterShako/media-automation-api.git

cd media-automation-api

Step 2: Create a Virtual Environment

python -m venv venv

Step 3: Activate the Virtual Environment (Windows PowerShell)

(Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ;
(.\venv\Scripts\Activate.ps1)

Step 4: Activate the Virtual Environment (Mac / Linux)

source venv/bin/activate

Step 5: Install All Dependencies

pip install fastapi uvicorn sqlmodel pymysql cryptography
openai-whisper streamlit requests

Step 6: Set Up MySQL Database (Run in MySQL Workbench)

CREATE DATABASE IF NOT EXISTS media_api;

USE media_api;

Step 7: Update Your MySQL Password in database.py

password = urllib.parse.quote_plus("YOUR_MYSQL_PASSWORD_HERE")

Step 8: Fix transcription_text Column (Run after first startup)

USE media_api;
ALTER TABLE mediatask MODIFY COLUMN transcription_text TEXT;

NOTE: FFmpeg must be installed and added to your system PATH before running the app. Download from gyan.dev (Windows) or install via brew install ffmpeg (Mac).

5. Minimal Working Example

Running the Application

Terminal 1: Start the API backend:

```
uvicorn main:app --reload # API
available at: http://127.0.0.1:8000
# Swagger docs at: http://127.0.0.1:8000/docs
```

Terminal 2: Start the Streamlit UI:

```
streamlit run ui.py # UI opens at:
http://localhost:8501
```

Core Transcription Function (services.py)

```
import whisper

def transcribe_audio(file_path: str, task: str = "transcribe") -> str:
# Load the Whisper model (base=fastest, medium=most accurate)
model = whisper.load_model("medium")

    # transcribe = keep original language | translate = convert to English     result
= model.transcribe(file_path, task=task)

    return result["text"]
```

API Upload Endpoint (main.py)

```
@app.post("/upload/") async def
upload_media(    file: UploadFile = File(...),    task: str
= Query(default="transcribe"),    session: Session =
Depends(get_session) ):    # Save file, run Whisper,
store result, return JSON    file_location =
f"temp_{file.filename}"    with open(file_location,
"wb") as buffer:        shutil.copyfileobj(file.file,
buffer)

   transcribed_text = transcribe_audio(file_location, task=task)
os.remove(file_location)
# clean up temp file
return {"text": transcribed_text, "filename": file.filename}
```

Expected Output

```
{
 "message": "Transcription complete!",
 "filename": "interview.wav",
 "task": "transcribe",
 "text": "Welcome to this course on Insurance and Law..."
}
```

6. AI Prompt Journal

Prompt 1	*How do I create a FastAPI endpoint that accepts a file upload and saves it to disk?*
Context	Initial scaffolding of the /upload/ endpoint
AI Summary	The AI provided a complete working endpoint using UploadFile
Evaluation	Helpful. Approximately 2 hours of reading FastAPI documentation is saved. The code worked on first attempt
Reflection	I learned that FastAPI uses Python type hints as the primary mechanism for defining what route accepts.
Prompt 2	*How do I connect FastAPI to MySQL using SQLModel and PyMySQL?*
Context	Setting up database.py with engine and session management
AI Summary	AI explained the create_engine() pattern, the difference between Session and AsyncSession
Evaluation	Very helpful. The explanation of dependency injection in FastAPI was particularly clear.
Reflection	Dependency injection (Depends()) is a powerful pattern, it automatically closes database sessions after each runtime.
Prompt 3	*I'm getting this error: pymysql.err.DataError: (1406, 'Data too long for column transcription_text at row 1'). How do I fix it?*
Context	After first successful transcription — the text was ~1,800 characters but MySQL VARCHAR defaults to 255
AI Summary	AI diagnosed the issue: SQL Model defaults Optional[str] to VARCHAR(255).
Evaluation	Extremely helpful. This was a blocking error, and the AI identified the root cause and provided fix
Reflection	Always specify column types explicitly when storing variable-length AI output, never assume VARCHAR(255)
Prompt 4	*RuntimeError: 'cryptography' package is required for sha256_password or caching_sha2_password auth methods*
Context	MySQL 8 authentication error on startup
AI Summary	AI explained that MySQL 8 changed its default authentication method from mysql_native_password to caching_
Evaluation	One-line fix: pip install cryptography.
Reflection	Database driver compatibility issues are common when upgrading MySQL versions. Always check auth method

Prompt 5	*How do I add a task parameter to my FastAPI endpoint so the Streamlit UI can choose between transcribe and translate modes?*
Context	Adding Swahili/English toggle feature
AI Summary	AI showed how to add task: str = Query(default='transcribe') as a query parameter, pass it through to the Whisper
Evaluation	Helpful. The solution was clean and required minimal changes across three files.
Reflection	FastAPI's Query() parameter is a clean way to add optional configuration to endpoints
Prompt 6	*How do I build a Streamlit UI with a dark theme, file uploader, audio preview, and download button for my transcription API?*
Context	Building the frontend ui.py
AI Summary	AI generated a complete Streamlit app with custom CSS for dark theming and st.file_uploader()
Evaluation	Very helpful. Streamlit's API is simple but knowing which components to use required guidance.
Reflection	Streamlit's st.columns() is a powerful layout tool. The entire UI was built in pure Python with no HTML knowledge
Prompt 7	*How do I upload my project to GitHub via the terminal and what files should I exclude?*
Context	Final project submission setup
AI Summary	AI walked through git init, .gitignore creation (excluding venv/, __pycache__/, temp_* files), git add/commit/push
Evaluation	Essential guidance. Without this, the venv folder (400MB+) would have been pushed to GitHub.
Reflection	Always create a .gitignore before the first commit. It is much harder to remove files from git history after they have been comitted

7. Common Issues & Fixes

Error: pymysql.err.DataError: Data too long for column 'transcription_text'

Cause: MySQL column defaults to VARCHAR(255). Long transcriptions exceed this limit.

Fix:

```
USE media_api;
ALTER TABLE mediatask MODIFY COLUMN transcription_text TEXT;
```

Note: Update models.py to use sa_column=Column(Text) to prevent recurrence.

Error: RuntimeError: 'cryptography' package is required for caching_sha2_password

Cause: MySQL 8 uses SHA2 authentication by default which requires RSA encryption support.

Fix: pip install

```
cryptography
```

Note: This is a one-time install. Restart uvicorn after installing.

Error: fatal: Unable to create '.git/index.lock': File exists

Cause: A previous git process was interrupted and left a lock file behind.

Fix:

```
Remove-Item .git/index.lock    # Windows
PowerShell rm .git/index.lock          # Mac / Linux
```

Note: This is safe to delete — it is just a stale lock file.

Error: Import 'streamlit' could not be resolved (VS Code warning)

Cause: VS Code is pointing to the system Python instead of the virtual environment.

Fix:

```
Ctrl+Shift+P > Python: Select Interpreter
> Choose: .\venv\Scripts\python.exe
```

Note: This is a display warning only. The code still runs correctly in the terminal.

Error: streamlit: command not found / uvicorn not recognized

Cause: Running commands outside the virtual environment.

Fix:

```
Always activate venv
first: .\venv\Scripts\Activate.ps1   #
Windows source venv/bin/activate      #
Mac/Linux
```

Note: Look for (venv) at the start of your terminal prompt to confirm activation.

Error: Cannot reach API. Make sure uvicorn is running on port 8000

Cause: The FastAPI backend is not running when Streamlit tries to call it.

Fix:

```
Open a second terminal, activate venv, and run:
uvicorn main:app --reload
```

Note: Both terminals must be running simultaneously — API in one, Streamlit in another.

Error: error: src refspec main does not match any

Cause: Trying to push before making a commit, or branch name mismatch.

Fix:

```
git add .
git commit -m "Initial
commit" git branch -M main
git push -u origin main
```

8. References

Official Documentation

Title	URL
FastAPI Official Docs	https://fastapi.tiangolo.com
OpenAI Whisper GitHub	https://github.com/openai/whisper
SQLModel Docs	https://sqlmodel.tiangolo.com
Streamlit Docs	https://docs.streamlit.io
MySQL 8.0 Reference	https://dev.mysql.com/doc/refman/8.0/en/
PyMySQL Docs	https://pymysql.readthedocs.io

FFmpeg Downloads

Title	URL
FFmpeg Official	https://ffmpeg.org/download.html
Windows Builds (gyan.dev)	https://www.gyan.dev/ffmpeg/builds/

Helpful Tutorials

Title	URL
FastAPI Crash Course	https://www.youtube.com/watch?v=0sOvCWFmrtA
Whisper AI — How It Works	https://openai.com/research/whisper
Streamlit	https://www.youtube.com/watch?v=JwSS70SZdyM

This Project

Title	URL
GitHub Repository	https://github.com/PeterShako/media-automation-api
AI Tool Used	https://claude.ai

Learning Reflections

This project was built over approximately one week using generative AI (Claude) as the primary learning tool. The following reflections summaries the key lessons from the experience.

AI as a Debugging Partner

The most valuable use of AI was not generating boilerplate code but diagnosing specific errors. Pasting error messages directly into the AI and asking, 'why is this happening?' consistently produced accurate root-cause analysis and working fixes in seconds.

Prompt Specificity Matters

Vague prompts like 'help me with FastAPI' produced generic answers. Specific prompts like 'How do I add a query parameter to a FastAPI POST endpoint and pass it to a service function?' produced immediately usable code.

Understanding Before Copying

When AI generated code, asking follow-up questions like 'why did you use yield instead of return here?' deepened understanding and prevented copy-paste errors when the context was slightly different.

AI Does Not Replace Documentation

AI helped bootstrap understanding, but official documentation was still necessary for edge cases, version-specific behavior, and production considerations. AI and Docs work best together.

Iterative Development

The project evolved through many small iterations, each error became a new prompt. This iterative AI-assisted debugging loop is significantly faster than traditional trial-and-error approaches.